

AI Agent Engineer — Technical Assessment

Agentic Search Intelligence System (LangChain / LangGraph DAG)

Version 2.0

1. Background & Context

We build AI visibility and search intelligence software that helps businesses understand and improve how they appear in AI-generated answers (ChatGPT, Gemini, Perplexity, AI Overviews, etc.) and traditional search engines. A core part of our product depends on multi-step data pipelines that query external search/data APIs, reason over the results, and produce structured intelligence for customers.

For this assessment, you will design and build a small but production-minded agentic system: a directed acyclic graph (DAG) of cooperating agents, orchestrated with LangChain/LangGraph, that answers a search-intelligence question end-to-end — from query planning through data retrieval to a final structured report.

We are less interested in a system that merely “works once” on the happy path, and much more interested in how you structure agent responsibilities, handle real-world API failure, and make the system observable — because that is exactly the kind of system you'd be building with us.

2. Objective

Build a multi-agent pipeline, orchestrated as a DAG, that:

- Takes a natural-language research question about a brand/keyword's search or AI visibility (e.g., “How does [Brand] show up in AI answers and search results for 'best project management software'?”).
- Plans the retrieval steps needed to answer it.
- Calls DataForSEO's AI/search APIs (e.g., SERP API, AI Search / AI Overview / LLM visibility endpoints) via correctly defined tool-calling schemas.
- Extracts and normalizes the relevant data.
- Synthesizes a final structured report/answer.

Each stage of this pipeline must be implemented as a separate agent/node with a single, atomic responsibility — no agent should both fetch data and also reason/summarize, for example.

3. System Requirements

3.1 DAG Orchestration

- Implement the pipeline as an explicit DAG using LangGraph (or an equivalent LangChain-based graph/DAG orchestration approach) — not a single monolithic chain or a linear if/else script.
- The graph should have clearly named nodes and edges, and conditional routing where appropriate (e.g., route to a fallback path if a retrieval node fails validation).
- Include a diagram (ASCII, Mermaid, or image) of your DAG in the README.

3.2 Atomic, Single-Responsibility Agents

Each agent/node should do one job only. At minimum, your DAG should include distinct agents for:

Agent	Sole Responsibility
Query Planner	Parse the user's question and decide which searches/API calls are needed
Search / Retrieval Agent(s)	Call the relevant DataForSEO endpoint(s) with correctly-formed tool calls
Extraction / Normalization Agent	Parse raw API responses into a clean, structured schema
Analysis / Synthesis Agent	Reason over normalized data to produce insights
Report Agent	Assemble the final structured output (JSON + human-readable summary)

Do not combine these responsibilities into fewer, “do-everything” agents — this is a primary grading criterion.

3.3 Correct Tool Calling

- Define tools using proper schemas (e.g., Pydantic models / JSON schema) with accurate types, descriptions, and required fields — not just free-text prompts asking the model to “call the API.”
- The LLM should decide when and with what arguments to call each tool; your code should validate the LLM's tool call arguments before executing the real API call.
- Handle malformed or partial tool-call arguments gracefully (e.g., missing required field) rather than crashing.

3.4 DataForSEO Integration

- Use DataForSEO's API for search/AI-visibility data (a free-tier/sandbox key, mock responses behind a clearly-marked flag, or a documented stub are all acceptable — state clearly in your README which mode you used).
- Wrap each DataForSEO call behind a clean tool interface (one tool per logical API call, not one giant “call DataForSEO” tool).

3.5 Failure Handling & Retries

- Implement retry logic with exponential backoff and jitter for transient API failures (timeouts, 429s, 5xxs).
- Distinguish retryable errors (network/timeout/rate-limit) from non-retryable ones (bad request, auth failure) and handle each appropriately.
- Set sane timeouts on all external calls.
- After exhausting retries, the DAG should degrade gracefully (e.g., route to a fallback node, return partial results with a clear error flag) rather than crashing the whole pipeline.
- Bonus: circuit-breaker behavior for a repeatedly-failing dependency.

3.6 Observability & Monitoring

- Structured logging (e.g., JSON logs) for every node execution: inputs (redacted where sensitive), outputs, duration, success/failure, retry count.

- Basic tracing across the DAG run so a single request can be followed node-by-node (LangSmith, OpenTelemetry, or a custom correlation-ID based trace log are all acceptable).
- Emit basic metrics: per-node latency, success/failure rate, and API call counts. A simple in-memory or logged summary at the end of a run is sufficient.
- README should explain what you'd add for production-grade observability if given more time.

4. API Specification

Implement the following endpoints for triggering and inspecting DAG runs. All endpoints should return JSON. Authentication is out of scope — no auth layer is required.

4.1 Business Profile Endpoints

POST /api/v1/profiles

Register a new brand/keyword profile. This is the entry point for the DAG pipeline.

Request body:

```
{
  "name": "Surfer SEO",
  "domain": "surferseo.com",
  "industry": "SEO Software",
  "description": "AI-powered SEO content optimization tool",
  "competitors": ["clearscope.io", "marketmuse.com", "frase.io"]
}
```

Response (201):

```
{
  "profile_uuid": "abc-123",
  "name": "Surfer SEO",
  "domain": "surferseo.com",
  "status": "created",
  "created_at": "2025-01-15T10:00:00Z"
}
```

GET /api/v1/profiles/{profile_uuid}

Retrieve a profile along with summary stats for it (total DAG runs, most recent run status, average opportunity score across discovered queries).

4.2 Pipeline Endpoints

POST /api/v1/profiles/{profile_uuid}/run

Trigger the full DAG for a profile. This is the core endpoint. The graph must execute in sequence — Query Planner → Search/Retrieval Agent(s) → Extraction/Normalization Agent → Analysis/Synthesis Agent → Report Agent — following whatever conditional routing and fallback paths you've defined.

The response should include:

- A pipeline run UUID
- Status (completed / failed / partial)

- Count of retrieval calls planned by the Query Planner
- Count of records successfully extracted and normalized
- Top insights from the Analysis/Synthesis Agent (with relevance/opportunity scores)
- The final structured report (JSON + human-readable summary) produced by the Report Agent
- Total tokens used (if available from your provider)

Performance note: A run may take 10–30 seconds depending on DataForSEO and LLM provider latency. You do not need to implement async/background processing — synchronous is fine. If you do implement async (e.g., Celery or another task queue), note it in your README as a bonus.

GET /api/v1/profiles/{profile_uuid}/queries

Return all sub-queries/retrieval calls planned for a profile's most recent run, sorted by opportunity score descending.

Supports query params:

- `?min_score=0.5` — filter by minimum opportunity score
- `?status=visible|not_visible|unknown` — filter by visibility status
- `?page=1&per_page=20` — pagination

Each query object should include:

- `query_text`
- `estimated_search_volume` (integer)
- `competitive_difficulty` (0–100)
- `opportunity_score` (float, 0–1, your formula)
- `domain_visible` (boolean)
- `visibility_position` (integer or null)
- `discovered_at` (ISO timestamp)

GET /api/v1/profiles/{profile_uuid}/recommendations

Return content recommendations synthesized by the Analysis/Synthesis Agent and assembled by the Report Agent.

Each recommendation should include:

- `recommendation_uuid`
- `target_query_uuid` (the query this addresses)
- `content_type` (e.g. `blog_post`, `landing_page`, `faq`)
- `title` (suggested content title)
- `rationale` (why this content addresses the gap)
- `target_keywords` (list of strings)
- `priority` (high / medium / low)

POST /api/v1/queries/{query_uuid}/recheck

Re-run the Search/Retrieval Agent (and the downstream Extraction/Normalization and Analysis/Synthesis nodes) for a single query. Returns updated data. Useful for re-checking a query after content has been published or after a prior run took a fallback path.

5. Deliverables

- Source code for the DAG-based agent system (Python, LangChain/LangGraph), including the API server and persistence layer described in Section 4 (profiles, runs, queries, and recommendations).
- README.md including:
 - Architecture overview + DAG diagram
 - Setup/run instructions
 - Explanation of agent responsibilities
 - How failures/retries are handled, with an example of a simulated failure
 - How observability works, with a sample trace/log excerpt
 - Known limitations and what you'd improve with more time
- Tests covering at minimum: one happy-path run, one simulated API failure with successful retry/fallback, and tool-call argument validation.
- A short .env.example documenting any required configuration (API keys, timeouts, retry limits, etc.).

6. Evaluation Criteria

Area	What we're looking for
Agent design	Each agent has a single, clearly-scoped responsibility; clean interfaces between nodes
DAG orchestration	Correct, non-linear graph structure with sensible routing/conditionals
Tool calling	Well-typed schemas, argument validation, correct invocation of DataForSEO tools
API design	Sensible REST resource model and status codes, correct request/response shapes, sound persistence design for profiles/runs/queries/recommendations, input validation
Resilience	Real retry/backoff logic, correct error classification, graceful degradation
Observability	Structured logs/traces that would actually help debug a production incident
Code quality	Readability, typing, testing, sensible project structure
Communication	Clarity of README and tradeoff explanations

7. Submission Guidelines

- Submit as a Git repository (zip or link) with clear commit history if possible.
- Include instructions to run locally with a single command where feasible (e.g., make run or python main.py).

- Time expectation: 5–7 hours. It is fine to stub or mock parts you don't have time to fully implement — document what you would do next.

This assessment supersedes prior Python Developer assessment versions for the AI Agent Engineer track. Contact the hiring team with any questions about scope or DataForSEO access.